```c
#include <stdio.h>

#define MAX 20

int n_states, n_symbols;
int transition[MAX][MAX];
int final_states[MAX], n_final;
int removed[MAX] = {0};

int is_final(int s) {
  for (int i = 0; i < n_final; i++) {
    if (final_states[i] == s) {
      return 1;
    }
  }
  return 0;
}

int equivalent(int s1, int s2) {
  if (is_final(s1) != is_final(s2)) {
    return 0;
  }
  for (int j = 0; j < n_symbols; j++) {
    if (transition[s1][j] != transition[s2][j]) {
      return 0;
    }
  }
  return 1;
}

void substitute_state(int old_state, int new_state) {
  for (int i = 0; i < n_states; i++) {
    for (int j = 0; j < n_symbols; j++) {
      if (transition[i][j] == old_state) {
        transition[i][j] = new_state;
      }
    }
  }
}

int main() {
  int i, j;

  printf("Enter number of states: ");
  scanf("%d", &n_states);

  printf("Enter number of input symbols: ");
  scanf("%d", &n_symbols);

  printf("Enter transition table (δ(qi, symbol) = qj):\n");
  for (i = 0; i < n_states; i++) {
    for (j = 0; j < n_symbols; j++) {
      printf("δ(q%d,%c) = ", i, 'a' + j);
      scanf("%d", &transition[i][j]);
    }
  }

  printf("Enter number of final states: ");
  scanf("%d", &n_final);

  printf("Enter final states: ");
  for (i = 0; i < n_final; i++) {
    scanf("%d", &final_states[i]);
  }

  for (i = 0; i < n_states; i++) {
```

```
    if (removed[i]) {
      continue;
    }
    for (j = i + 1; j < n_states; j++) {
      if (!removed[j] && equivalent(i, j)) {
        printf("q%d and q%d are equivalent (same group) → Eliminating q%d\n", i,
               j, j);
        substitute_state(j, i);
        removed[j] = 1;
      }
    }
  }

  printf("\nMinimized DFA Transition Table:\n");

  printf("State\t");
  for (j = 0; j < n_symbols; j++) {
    printf("%c\t", 'a' + j);
  }
  printf("\n");

  for (i = 0; i < n_states; i++) {
    if (!removed[i]) {
      printf("q%d\t", i);
      for (j = 0; j < n_symbols; j++) {
        printf("q%d\t", transition[i][j]);
      }
      printf("\n");
    }
  }

  return 0;
}
```